Student Performance Prediction

using Machine Learning

Name: Mukhammadsaiid Norbaev

Student ID: NOR22597897

Module: Data Science

Coursework: Coursework 2

Submission: 30 April 2026

Table of Contents

1. Introduction

This report presents the implementation and critical evaluation of machine learning models to predict student academic outcomes, building directly on the dataset analysis and method proposal developed in Coursework 1 (CW1). The three problems identified in CW1 are each addressed through the implementation described in this report:

- **Problem 1:** Identify students at risk of failing to enable early intervention.
- **Problem 2:** Analyse which factors most influence student performance.
- **Problem 3:** Predict binary pass/fail outcomes from student data.

Naive Bayes was proposed in CW1 as the primary classification technique and is implemented here as planned. Logistic Regression is added as an extension model to provide a comparison and to extract feature importance insights through model coefficients — directly addressing the CW1 feedback to analyse variable effects on the target outcome.

2. Dataset and Preprocessing Summary

The dataset, sourced from Kaggle (Student Performance Factors), contains 6,607 student records across 20 attributes — a mix of numerical features (Hours_Studied, Attendance, Sleep_Hours, Previous_Scores) and categorical features (Motivation_Level, School_Type, Gender, Family_Income). The following preprocessing steps were applied:

```
# Check structure: column names, data types, and missing value counts
# This confirms which columns have missing values that need handling
df.info()
```

✓ 0.0s

```
<class 'pandas.DataFrame'>
RangeIndex: 6607 entries, 0 to 6606
Data columns (total 21 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   Hours_Studied               6607 non-null   int64
 1   Attendance                  6607 non-null   int64
 2   Parental_Involvement        6607 non-null   str
 3   Access_to_Resources         6607 non-null   str
 4   Extracurricular_Activities  6607 non-null   str
 5   Sleep_Hours                 6607 non-null   int64
 6   Previous_Scores             6607 non-null   int64
 7   Motivation_Level            6607 non-null   str
 8   Internet_Access             6607 non-null   str
 9   Tutoring_Sessions           6607 non-null   int64
 10  Family_Income               6607 non-null   str
 11  Teacher_Quality             6529 non-null   str
 12  School_Type                 6607 non-null   str
 13  Peer_Influence              6607 non-null   str
 14  Physical_Activity           6607 non-null   int64
 15  Learning_Disabilities       6607 non-null   str
 16  Parental_Education_Level    6517 non-null   str
 17  Distance_from_Home          6540 non-null   str
 18  Gender                      6607 non-null   str
 19  Exam_Score                  6607 non-null   int64
 20  Pass                        6607 non-null   int64
dtypes: int64(8), str(13)
memory usage: 1.1 MB
```

```
# Display basic statistics for all numerical columns
# Useful for understanding feature ranges before applying Min-Max normalisation
df.describe()
```

✓ 0.0s

	Hours_Studied	Attendance	Sleep_Hours	Previous_Scores	Tutoring_Sessions	Physical_Activity	Exam_Score	Pass
count	6607.000000	6607.000000	6607.00000	6607.000000	6607.000000	6607.000000	6607.000000	6607.000000
mean	19.975329	79.977448	7.02906	75.070531	1.493719	2.967610	67.235659	0.245951
std	5.990594	11.547475	1.46812	14.399784	1.230570	1.031231	3.890456	0.430682
min	1.000000	60.000000	4.00000	50.000000	0.000000	0.000000	55.000000	0.000000
25%	16.000000	70.000000	6.00000	63.000000	1.000000	2.000000	65.000000	0.000000
50%	20.000000	80.000000	7.00000	75.000000	1.000000	3.000000	67.000000	0.000000
75%	24.000000	90.000000	8.00000	88.000000	2.000000	4.000000	69.000000	0.000000
max	44.000000	100.000000	10.00000	100.000000	8.000000	6.000000	101.000000	1.000000

Figure 1: Dataset structure — 6,607 records, 21 columns, missing values in 3 columns

- **Missing value imputation:** Teacher_Quality (78 missing), Parental_Education_Level (90 missing), and Distance_from_Home (67 missing) were imputed using the mode (most frequent category), preserving all 6,607 records.
- **Duplicate removal:** Duplicate rows were checked and removed to prevent the model from overfitting to repeated entries.
- **Pass variable redefinition:** In CW1, a threshold of 50 was used. However, since all exam scores are 55 or above, this classified every student as pass — making the

dataset completely unsuitable for classification. The threshold was corrected to 70, producing 4,982 fail (75.3%) and 1,625 pass (24.7%). This class imbalance is critical to acknowledge: a naive model that always predicts 'fail' would already achieve 75.3% accuracy, so any meaningful model must substantially exceed this baseline.

- **Data leakage prevention:** Exam_Score was removed from the feature set because Pass is derived directly from it. Retaining it would cause artificially inflated accuracy as the model would reverse-engineer the threshold rather than learning from student behaviour and environmental features.
- **One-hot encoding:** Categorical variables were converted to binary dummy columns, applied separately to training and test sets to prevent leakage.
- **Min-Max normalisation:** All numerical features were scaled to [0, 1]. The scaler was fitted exclusively on training data and then applied to test data, ensuring the test distribution remains unseen during preprocessing.

```python
# Redefine Pass variable: 1 = pass (score >= 70), 0 = fail (score < 70)
# Threshold of 50 used in CW1 was incorrect as all scores were above 50
# Threshold of 70 creates a meaningful binary classification problem
df["Pass"] = (df["Exam_Score"] >= 70).astype(int)

# Inspect the class distribution to understand potential imbalance
# Important: a heavily skewed distribution affects how we interpret accuracy
print("Class Distribution (Pass variable):")
print(df["Pass"].value_counts())
print(f"\nClass balance: {df['Pass'].value_counts(normalize=True).round(3) * 100}%")

# Visualise class distribution
df["Pass"].value_counts().plot(kind='bar', color=['salmon', 'steelblue'], edgecolor='black')
plt.title("Class Distribution: Pass vs Fail")
plt.xlabel("Pass (1) / Fail (0)")
plt.ylabel("Count")
plt.xticks(rotation=0)
plt.show()
```

✓ 0.0s

```
Class Distribution (Pass variable):
Pass
0    4982
1    1625
Name: count, dtype: int64

Class balance: Pass
0    75.4
1    24.6
Name: proportion, dtype: float64%
```

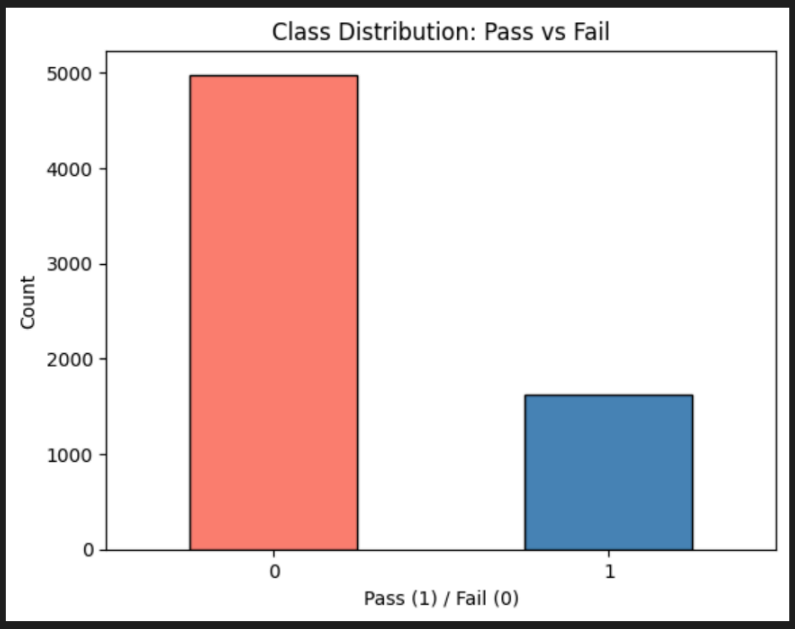
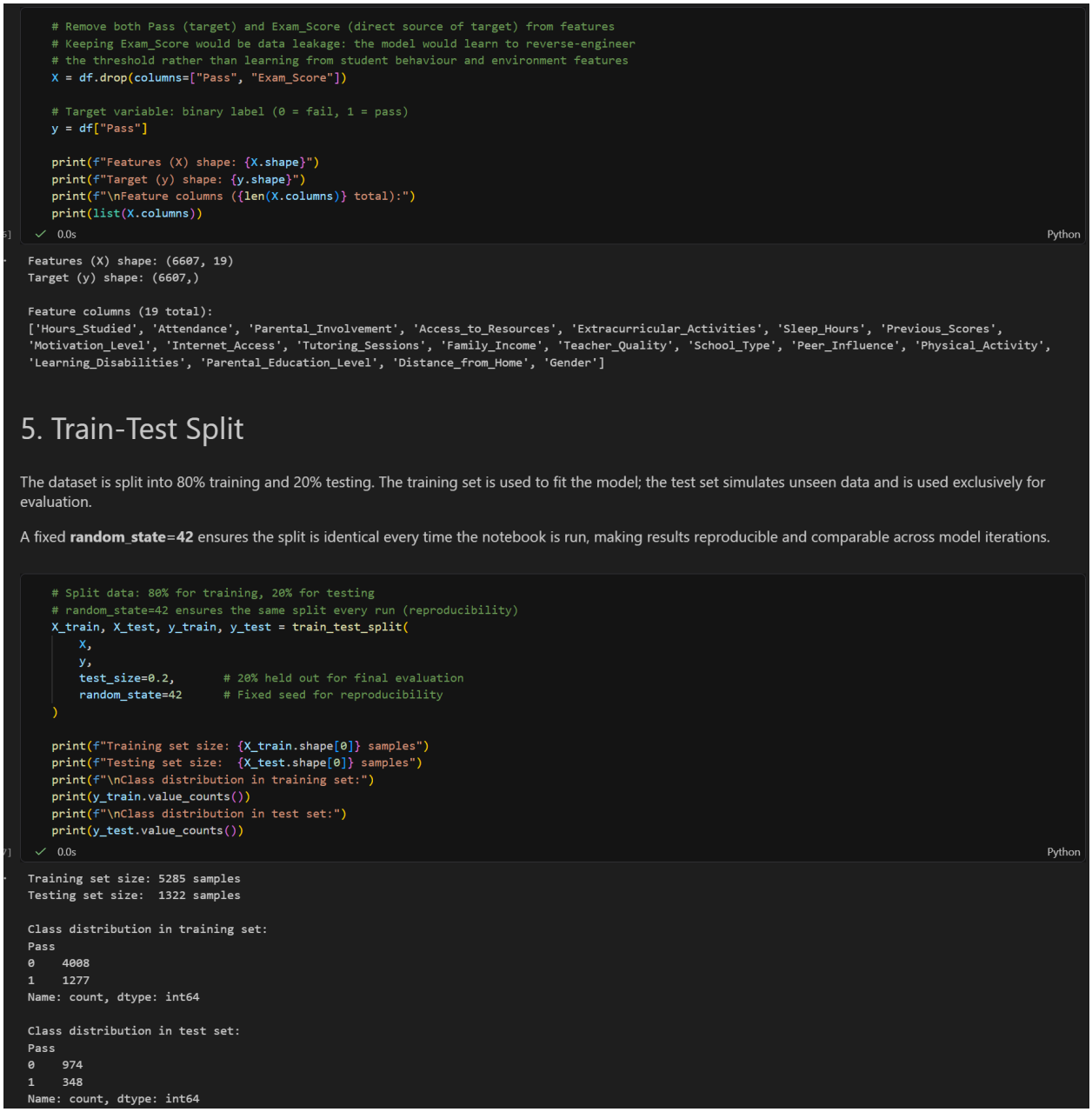


Figure 2: Class distribution after threshold correction to 70 — 75.4% fail, 24.6% pass

3. Implementation

3.1 Data Preparation and Train-Test Split

The dataset was loaded using pandas and split into features (X) and target (y). Exam_Score and Pass were both removed from X to prevent data leakage. An 80/20 train-test split was applied using random_state=42 to guarantee reproducibility. The training set (5,285 samples) was used exclusively for model fitting; the test set (1,322 samples) was held out for final evaluation.



```
# Remove both Pass (target) and Exam_Score (direct source of target) from features
# Keeping Exam_Score would be data leakage: the model would learn to reverse-engineer
# the threshold rather than learning from student behaviour and environment features
X = df.drop(columns=["Pass", "Exam_Score"])

# Target variable: binary label (0 = fail, 1 = pass)
y = df["Pass"]

print(f"Features (X) shape: {X.shape}")
print(f"Target (y) shape: {y.shape}")
print(f"\nFeature columns ({len(X.columns)} total):")
print(list(X.columns))
```

✓ 0.0s Python

```
Features (X) shape: (6607, 19)
Target (y) shape: (6607,)

Feature columns (19 total):
['Hours_Studied', 'Attendance', 'Parental_Involvement', 'Access_to_Resources', 'Extracurricular_Activities', 'Sleep_Hours', 'Previous_Scores',
'Motivation_Level', 'Internet_Access', 'Tutoring_Sessions', 'Family_Income', 'Teacher_Quality', 'School_Type', 'Peer_Influence', 'Physical_Activity',
'Learning_Disabilities', 'Parental_Education_Level', 'Distance_from_Home', 'Gender']
```

5. Train-Test Split

The dataset is split into 80% training and 20% testing. The training set is used to fit the model; the test set simulates unseen data and is used exclusively for evaluation.

A fixed **random_state=42** ensures the split is identical every time the notebook is run, making results reproducible and comparable across model iterations.

```
# Split data: 80% for training, 20% for testing
# random_state=42 ensures the same split every run (reproducibility)
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,       # 20% held out for final evaluation
    random_state=42      # Fixed seed for reproducibility
)

print(f"Training set size: {X_train.shape[0]} samples")
print(f"Testing set size:  {X_test.shape[0]} samples")
print(f"\nClass distribution in training set:")
print(y_train.value_counts())
print(f"\nClass distribution in test set:")
print(y_test.value_counts())
```

✓ 0.0s Python

```
Training set size: 5285 samples
Testing set size:  1322 samples

Class distribution in training set:
Pass
0    4008
1    1277
Name: count, dtype: int64

Class distribution in test set:
Pass
0    974
1    348
Name: count, dtype: int64
```

Figure 3: Feature selection (removing Exam_Score to prevent leakage) and 80/20 train-test split

3.2 Encoding and Scaling

Categorical variables were one-hot encoded using pd.get_dummies(), applied independently to training and test sets. Column alignment (.align() with join='left') ensured both sets had identical columns after encoding. Column names were saved after encoding but before scaling — Min-Max scaling converts the DataFrame to a NumPy array, so saving column names at this point ensures correct feature-to-coefficient mapping in the Logistic Regression analysis.

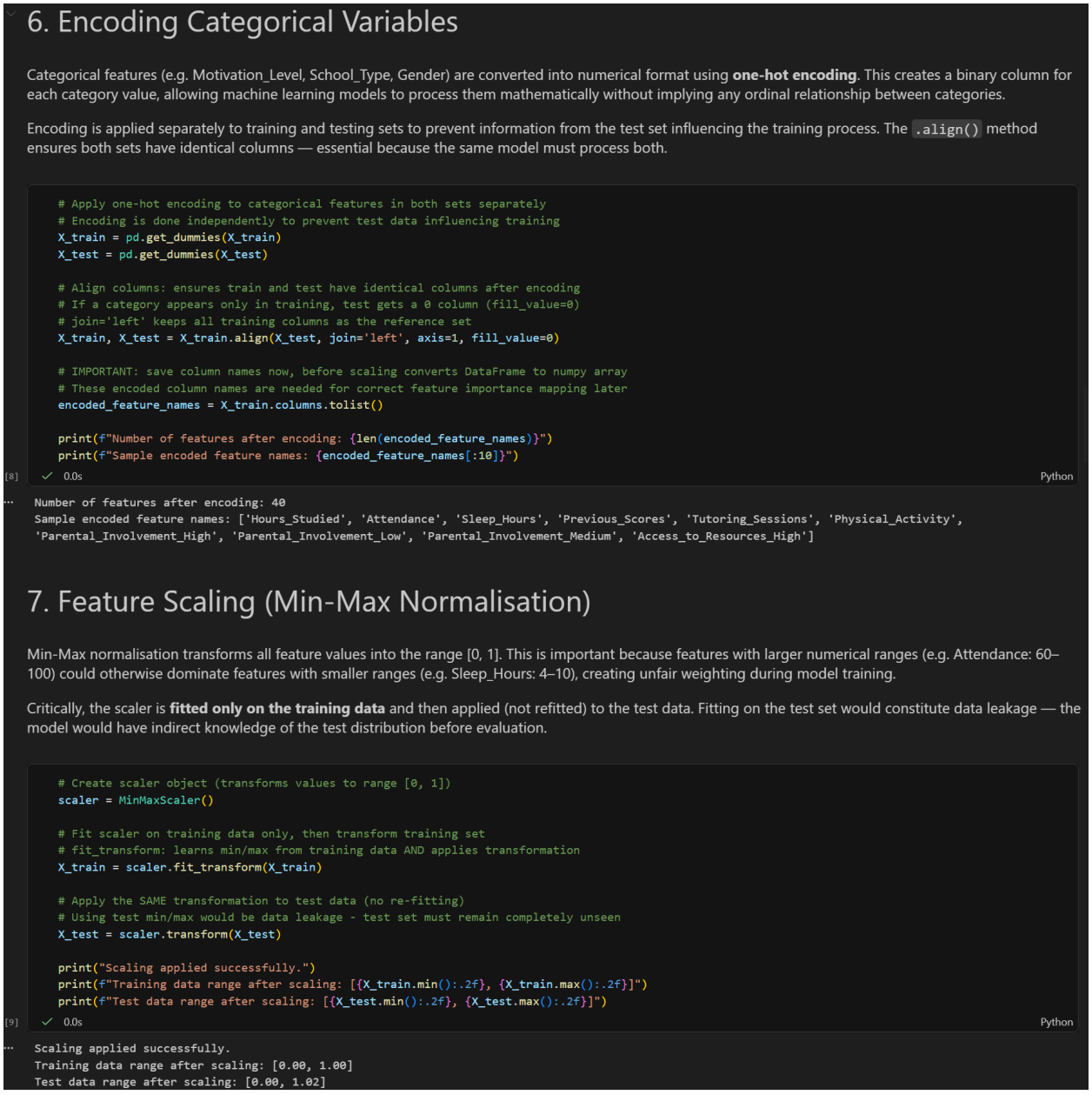


Figure 4: One-hot encoding and Min-Max normalisation applied to training and test sets separately

3.3 Naive Bayes Model

Gaussian Naive Bayes (GaussianNB) was implemented as the primary model proposed in CW1. It was trained on the preprocessed training data and evaluated on the held-out test set. The Gaussian variant is appropriate here because features are continuous after Min-Max scaling.

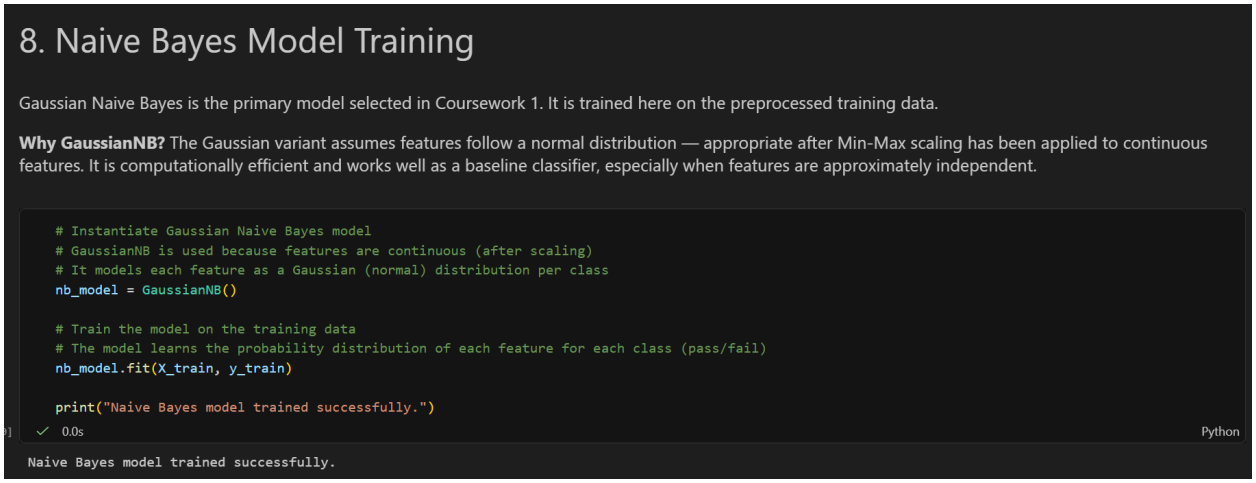


Figure 5: Naive Bayes model training using GaussianNB

Results — Accuracy: 88.1%. While this appears strong, it must be contextualised against the 75.3% majority-class baseline. The model achieves a genuine ~13 percentage point improvement, demonstrating it has learned meaningful patterns. However, class-level metrics reveal important nuance:

- **Class 0 (Fail) — Precision: 0.92, Recall: 0.91, F1: 0.92.** The model correctly identifies 890 of 974 failing students.
- **Class 1 (Pass) — Precision: 0.77, Recall: 0.79, F1: 0.78.** Performance is weaker for the minority class, with 84 false positives (fail students predicted as pass). In an educational context, **false positives are the most harmful error type** — a student predicted to pass who actually fails receives no intervention or support.

The performance gap between classes reflects the feature independence assumption in Naive Bayes. In reality, Hours_Studied and Attendance are correlated — students who study more also tend to attend more. This violated assumption reduces the model's ability to correctly classify the minority (pass) class.

10. Naive Bayes – Model Evaluation

Model performance is evaluated using three complementary metrics:

- **Accuracy**: Overall proportion of correct predictions. However, given the class imbalance (75% fail, 25% pass), a model that always predicts 'fail' would already achieve ~75% accuracy without learning anything. Accuracy alone is therefore insufficient.
- **Precision**: Of all students predicted to pass, how many actually did? Important for avoiding false positives (predicting pass for students who will fail).
- **Recall**: Of all students who actually passed, how many did the model correctly identify? Important for avoiding false negatives (missing students who need intervention).
- **F1-score**: Harmonic mean of precision and recall — a balanced single-number summary per class.

```
[12]  ✓ 0.0s                                                                    Python
```

```python
# Calculate overall accuracy
nb_accuracy = accuracy_score(y_test, y_pred_nb)
print(f"Naive Bayes Accuracy: {nb_accuracy:.4f} ({nb_accuracy*100:.1f}%)")

# Confusion matrix: shows the breakdown of correct and incorrect predictions per class
# Rows = actual class, Columns = predicted class
# [TN, FP]  = fail correctly predicted, fail incorrectly predicted as pass
# [FN, TP]  = pass incorrectly predicted as fail, pass correctly predicted
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_nb))

# Full classification report: precision, recall, and F1 broken down per class
# This is more informative than accuracy for imbalanced datasets
print("\nClassification Report:")
print(classification_report(y_test, y_pred_nb))
```

```
Naive Bayes Accuracy: 0.8812 (88.1%)

Confusion Matrix:
[[890  84]
 [ 73 275]]

Classification Report:
              precision    recall  f1-score   support

           0       0.92      0.91      0.92       974
           1       0.77      0.79      0.78       348

    accuracy                           0.88      1322
   macro avg       0.85      0.85      0.85      1322
weighted avg       0.88      0.88      0.88      1322
```

Figure 6: Naive Bayes accuracy, confusion matrix, and per-class classification report

11. Naive Bayes – Confusion Matrix Visualisation

The confusion matrix is visualised as a heatmap for clearer interpretation. Each cell shows:

- **Top-left (TN)**: Fail correctly predicted as fail
- **Top-right (FP)**: Fail incorrectly predicted as pass — students who will fail but receive no intervention
- **Bottom-left (FN)**: Pass incorrectly predicted as fail — students flagged unnecessarily
- **Bottom-right (TP)**: Pass correctly predicted as pass

In an educational context, **false positives (FP) are the most harmful error** — a student predicted to pass who actually fails would receive no support.

```
# Create confusion matrix heatmap for Naive Bayes
cm_nb = confusion_matrix(y_test, y_pred_nb)

plt.figure(figsize=(6, 4))
sns.heatmap(
    cm_nb,
    annot=True,          # Show numbers in each cell
    fmt="d",             # Integer format
    cmap="Blues",        # Colour scheme
    xticklabels=["Predicted Fail", "Predicted Pass"],
    yticklabels=["Actual Fail", "Actual Pass"]
)
plt.title("Naive Bayes Confusion Matrix")
plt.tight_layout()
plt.show()
```

[13] ✓ 0.0s Python

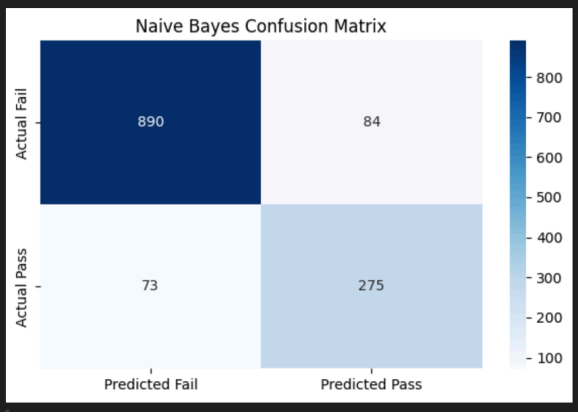


Figure 7: Naive Bayes confusion matrix — more errors on the pass (minority) class

3.4 Logistic Regression Model (Extension)

Logistic Regression was implemented as an extension to compare against Naive Bayes and to extract feature-level insights through model coefficients. It was trained on the identical preprocessed training data to ensure a fair comparison.

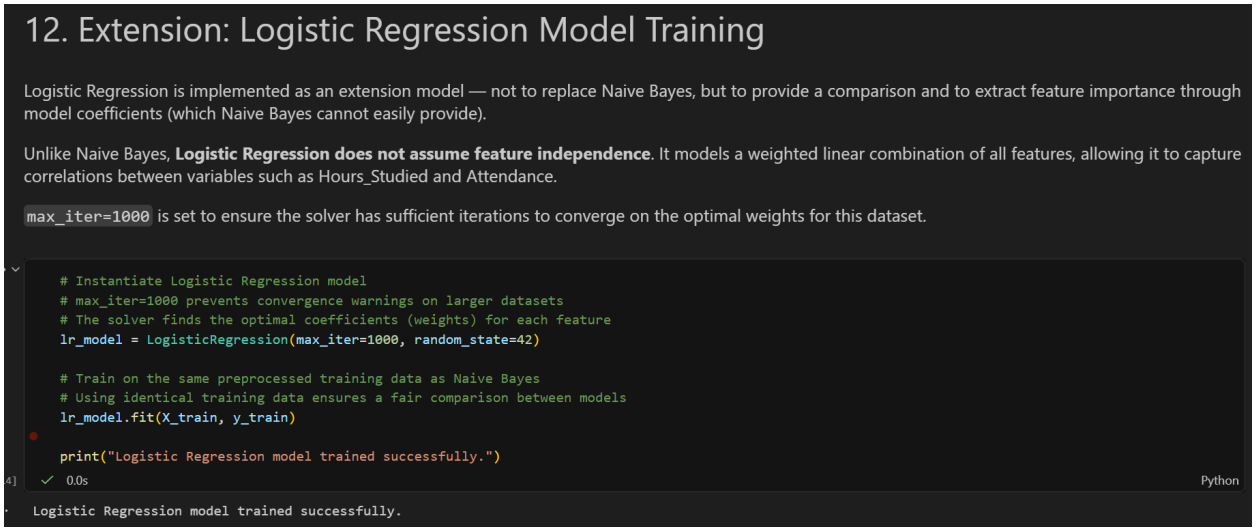


Figure 8: Logistic Regression model training

Results — Accuracy: 97.5%. The substantial gap over Naive Bayes (9.4 percentage points) is attributable to two factors: (1) Logistic Regression does not assume feature independence, allowing it to model correlated variables effectively; and (2) the narrow exam score distribution (55–101, median 67) creates a tight linear boundary near the threshold of 70, which the linear decision boundary of Logistic Regression exploits efficiently. This high accuracy should be interpreted with appropriate caution — it may reflect the structured nature of this dataset rather than guaranteed generalisability to real student populations.

Class-level recall for the minority class improved substantially: **Recall (Pass) = 0.92**, compared to 0.79 for Naive Bayes. Only 28 passing students were misclassified as fail, and only 5 failing students were predicted to pass — a major improvement in the most harmful error type.

14. Logistic Regression – Model Evaluation

The same evaluation metrics are applied to Logistic Regression results. The accuracy gap between the two models (if substantial) warrants critical examination — an unusually high accuracy might suggest the model is exploiting linear separability in the data (scores clustered near the threshold) rather than truly learning generalisable patterns.

```
# Calculate Logistic Regression accuracy
lr_accuracy = accuracy_score(y_test, y_pred_lr)
print(f"Logistic Regression Accuracy: {lr_accuracy:.4f} ({lr_accuracy*100:.1f}%)")

# Confusion matrix for Logistic Regression
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_lr))

# Detailed classification report: compare class-level metrics to Naive Bayes
# Pay attention to the minority class (Pass=1) recall - can the model find students who pass?
print("\nClassification Report:")
print(classification_report(y_test, y_pred_lr))
```

16] ✓ 0.0s Python

```
Logistic Regression Accuracy: 0.9750 (97.5%)

Confusion Matrix:
[[969   5]
 [ 28 320]]

Classification Report:
              precision    recall  f1-score   support

           0       0.97      0.99      0.98       974
           1       0.98      0.92      0.95       348

    accuracy                           0.98      1322
   macro avg       0.98      0.96      0.97      1322
weighted avg       0.98      0.98      0.97      1322
```

Figure 9: Logistic Regression accuracy, confusion matrix, and per-class classification report

16. Logistic Regression – Confusion Matrix Visualisation

The confusion matrix for Logistic Regression is visualised using the same format as Naive Bayes, enabling direct visual comparison. Notably fewer false positives and false negatives are expected given the higher overall accuracy.

```
# Create confusion matrix heatmap for Logistic Regression
cm_lr = confusion_matrix(y_test, y_pred_lr)

plt.figure(figsize=(6, 4))
sns.heatmap(
    cm_lr,
    annot=True,
    fmt="d",
    cmap="Purples",       # Different colour to visually distinguish from NB heatmap
    xticklabels=["Predicted Fail", "Predicted Pass"],
    yticklabels=["Actual Fail", "Actual Pass"]
)
plt.title("Logistic Regression Confusion Matrix")
plt.tight_layout()
plt.show()
```

0.0s

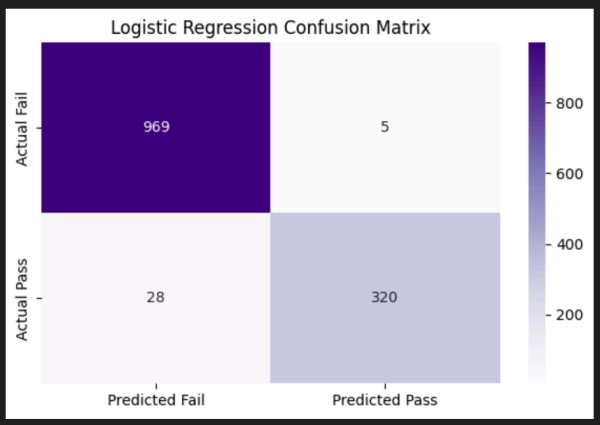


Figure 10: Logistic Regression confusion matrix — very few misclassifications across both classes

3.5 Feature Importance Analysis

Logistic Regression provides a coefficient for each input feature. A higher positive coefficient means the feature strongly increases the probability of predicting 'pass'. The table below shows the top 10 most influential features:

#	Feature	Coefficient	Interpretation
1	**Hours_Studied**	+14.59	Strongest predictor — most impactful feature overall
2	**Attendance**	+10.09	Strong predictor — closely linked to engagement
3	Extracurricular_Activities	+4.62	Moderate positive effect on pass probability
4	Access_to_Resources	+3.13	Moderate positive effect
5	Sleep_Hours	+1.58	Modest positive effect
6	Tutoring_Sessions	+1.36	Modest positive effect

7	Previous_Scores	+1.22	Weaker than expected — compressed score range reduces discriminative power
8	Physical_Activity	+0.73	Weak positive effect
9	Gender	+0.70	Minimal effect
10	Peer_Influence	+0.35	Minimal effect

Table 1: Top 10 feature importances ranked by Logistic Regression coefficient magnitude

Hours_Studied (14.59) and Attendance (10.09) are the dominant predictors, confirming the CW1 hypothesis that active engagement behaviours drive academic outcomes. Notably, Previous_Scores (1.22) has a weaker effect than expected — the compressed score range (50–100) reduces its discriminative power relative to behavioural features. This directly addresses Problem 2 from CW1 and the feedback to analyse variable effects on the target.

```
# Visualise top 15 feature importances as a horizontal bar chart
# Colour-coded: positive coefficients (increase pass probability) vs negative (decrease)
top_features = feature_importance.head(15)
colors = ['steelblue' if c > 0 else 'salmon' for c in top_features["Coefficient"]]

plt.figure(figsize=(10, 6))
plt.barh(top_features["Feature"], top_features["Coefficient"], color=colors, edgecolor='black')
plt.axvline(x=0, color='black', linewidth=0.8)  # Reference line at zero
plt.title("Top 15 Feature Importances (Logistic Regression Coefficients)")
plt.xlabel("Coefficient Value (positive = increases pass probability)")
plt.ylabel("Feature")
plt.tight_layout()
plt.show()
```

✓ 0.0s

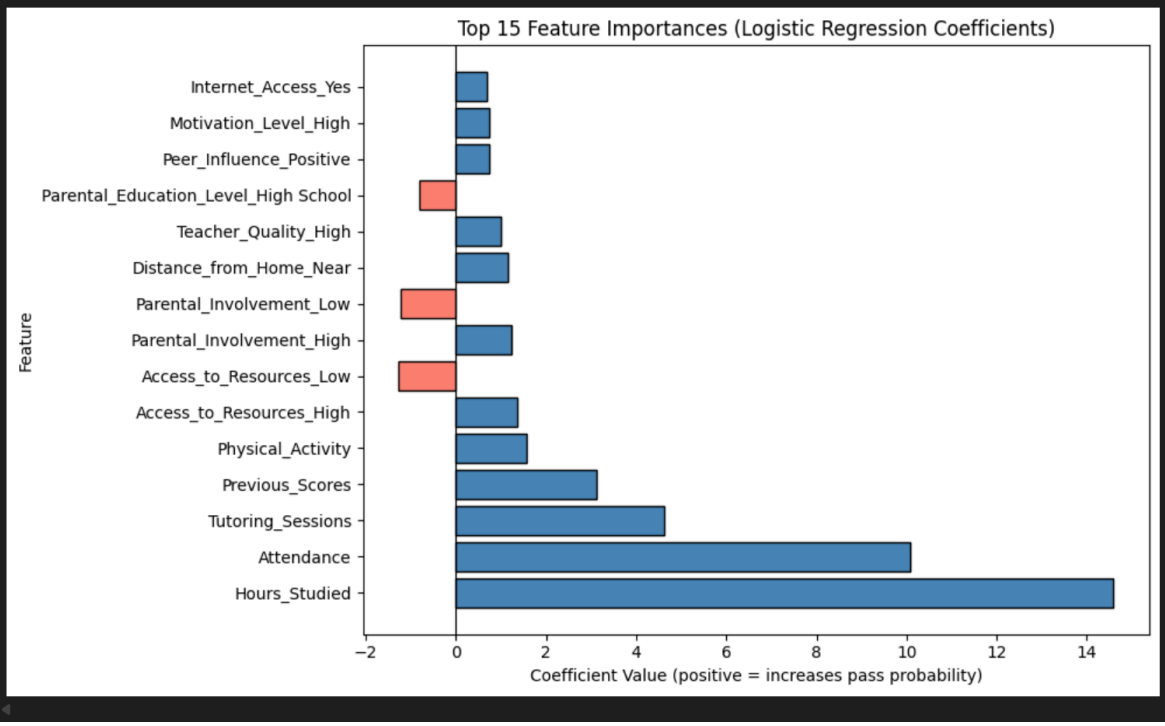


Figure 11: Feature importance — Hours_Studied and Attendance dominate the prediction

3.6 Model Comparison Summary

Both models are compared below across all key evaluation metrics. Logistic Regression outperforms Naive Bayes across every metric, with the greatest improvement seen in the minority class (Pass) recall — the most critical metric for educational intervention.

Metric	Naive Bayes	Logistic Regression
Accuracy	88.1%	97.5%
Precision – Class 0 (Fail)	0.92	0.97
Recall – Class 0 (Fail)	0.91	0.99
F1-Score – Class 0 (Fail)	0.92	0.98
Precision – Class 1 (Pass)	0.77	0.98
Recall – Class 1 (Pass)	0.79	0.92
F1-Score – Class 1 (Pass)	0.78	0.95
Macro Average F1	**0.85**	**0.97**

Table 2: Side-by-side model comparison across all key metrics

```python
# Visualise model comparison as a grouped bar chart
metrics = ["Accuracy", "Precision (macro)", "Recall (macro)", "F1-Score (macro)"]
nb_scores = comparison[comparison["Model"] == "Naive Bayes"][metrics].values[0]
lr_scores = comparison[comparison["Model"] == "Logistic Regression"][metrics].values[0]

x = np.arange(len(metrics))
width = 0.35

fig, ax = plt.subplots(figsize=(10, 5))
bars1 = ax.bar(x - width/2, nb_scores, width, label='Naive Bayes', color='steelblue', edgecolor='black')
bars2 = ax.bar(x + width/2, lr_scores, width, label='Logistic Regression', color='salmon', edgecolor='black')

# Add value labels on bars
for bar in bars1:
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
            f'{bar.get_height():.3f}', ha='center', va='bottom', fontsize=9)
for bar in bars2:
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
            f'{bar.get_height():.3f}', ha='center', va='bottom', fontsize=9)

ax.set_xlabel("Metric")
ax.set_ylabel("Score")
ax.set_title("Model Performance Comparison: Naive Bayes vs Logistic Regression")
ax.set_xticks(x)
ax.set_xticklabels(metrics)
ax.set_ylim(0.7, 1.05)
ax.legend()
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```

0.0s Python

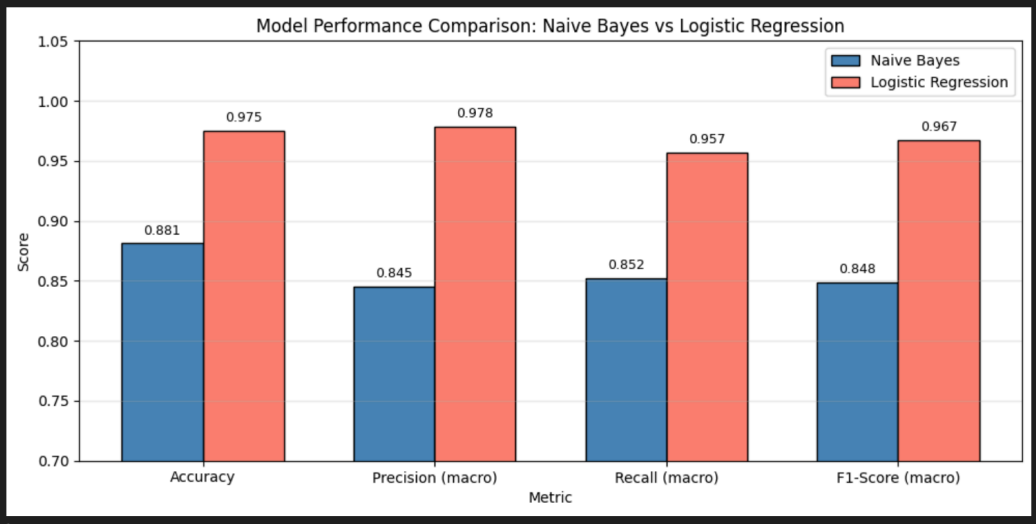


Figure 12: Model performance comparison — Logistic Regression outperforms Naive Bayes on all metrics

4. Reflection

4.1 Connection to Coursework 1 Goals

The three problems defined in CW1 have all been addressed in this implementation:

- **Problem 1 — Identify at-risk students:** Both models flag students with low Hours_Studied and Attendance as high-risk. Logistic Regression reduces false

positives to just 5 out of 1,322 test samples, making it highly reliable for real-world intervention targeting.

- **Problem 2 — Analyse factor influence:** Logistic Regression coefficients confirm Hours_Studied and Attendance as the most influential predictors, providing institutions with clear, actionable targets for intervention.
- **Problem 3 — Predict pass/fail:** Both models substantially exceed the 75.3% majority-class baseline — Naive Bayes by 12.8 percentage points, Logistic Regression by 22.2 percentage points.

4.2 Model Choice Evaluation

The selection of Naive Bayes in CW1 was a reasonable starting point — it is simple, efficient, and interpretable. The implementation revealed that its feature independence assumption is violated by this dataset, as Hours_Studied and Attendance are correlated. This likely explains its weaker minority-class recall compared to Logistic Regression.

The class imbalance (75% fail, 25% pass) was the most significant challenge encountered. The initial threshold of 50 produced a degenerate dataset; correcting this to 70 was critical. Even after correction, standard accuracy is a misleading metric — per-class precision and recall are essential for proper evaluation in imbalanced classification problems.

4.3 Limitations and Future Improvements

- **Class imbalance:** SMOTE (Synthetic Minority Oversampling Technique) or setting class_weight='balanced' in Logistic Regression could improve minority class recall further.
- **Continuous prediction:** Using regression to predict Exam_Score directly as a continuous value would provide more nuanced insights than binary classification.
- **Cross-validation:** Using k-fold cross-validation rather than a single train-test split would provide more robust accuracy estimates and reduce split-dependent variability.
- **Additional models:** Random Forest or Support Vector Machines could be tested as further comparison models.

5. Real-World Deployment

The trained Logistic Regression model could be integrated as a risk-flagging component within a Student Information System (SIS). At the start of each academic term, student data — including attendance records, self-reported study hours, and access to resources — would be collected and passed through the preprocessing pipeline to generate a binary risk score per student.

Students predicted to fail (class 0) would be automatically flagged for early intervention: referral to tutoring services, pastoral care, or targeted academic support programmes. This directly realises the goal of Problem 1 from CW1.

The model would require periodic retraining as new cohort data becomes available, since student populations and score distributions may shift over time. Before deployment, data privacy compliance — including GDPR requirements applicable to UK educational institutions — would need to be addressed, covering informed consent for data use, data minimisation, and the right of students to understand and challenge automated decisions that affect them.

6. References

1. Lainguyn123 (2023). Student Performance Factors. Kaggle. Available at: https://www.kaggle.com/datasets/lainguyn123/student-performance-factors [Accessed April 2026].
2. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, pp.2825–2830.
3. James, G., Witten, D., Hastie, T. and Tibshirani, R. (2013). An Introduction to Statistical Learning with Applications in R. New York: Springer.
4. Mitchell, T.M. (1997). Machine Learning. New York: McGraw-Hill.
5. McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, pp.51–56.